

PROGRAMACIÓN AVANZADA

Base de datos – Parte I



Ing. Matias Macrino

BASE DE DATOS.

- ▶ Conceptos básicos:

- ▶ Las base de datos, son una colección ordenada de información.
- ▶ Se pueden clasificar en:
 - ▶ Jerárquicas
 - ▶ Transaccionales
 - ▶ Relacionales

BASE DE DATOS.

- ▶ Conceptos básicos:
- ▶ *BD Transaccionales:*
 - ▶ Tienen por objetivo el envío y recepción de datos a grandes velocidades
 - ▶ Ejemplo: Traspaso de dinero entre cuentas bancarias.
- ▶ *BD Relacionales:*
 - ▶ Modelo que se utiliza actualmente para representas problemas reales y administrar datos dinámicamente.
 - ▶ Ejemplos: MS SQL Server, PostgreSQL, MySQL.

BASE DE DATOS

- ▶ Conceptos básicos:
- ▶ *Sistema gestor de base de datos:*
 - ▶ *Sistema que permite almacenar, modificar y extraer información de una base de datos, además proporciona herramientas para añadir, borrar, modificar y analizar los datos.*
- ▶ Para bases relacionales se encuentran los siguientes clientes para SGBD:
 - ▶ MySQL WorkBench (SGBD MySQL)
 - ▶ SQL Management Studio (SGBD MS SQL Server)
 - ▶ SQL Developer (SGBD Oracle)
 - ▶ Aqua Data Studio (SGBD MS SQL Server, SGBD MySQL, SGBD Oracle)

BASE DE DATOS

- ▶ Base de datos relacionales:

- ▶ Representación lógica de datos que permite que la información pueda ser accedida sin considerar su estructura física.
- ▶ *Los datos son almacenados en “tablas”.*
 - ▶ Se componen de filas y éstas por columnas, sobre cuales se almacenan los valores.
 - ▶ Cada columna representa un atributo diferente.
 - ▶ La columna que identifica unívocamente cada fila, se declara clave primaria.
 - ▶ Cada fila representa un registro.
- ▶ *Para consultar la información se desarrollan consultas sobre las tablas con SQL (Structured Query Language) los que se divide en DDL (Data Definition Language) y DML (Data Manipulation Language)*

JDBC – JAVA DATABASE CONNECTIVITY

- ▶ En java disponemos de un API especial basada en estándares que permite desarrollar aplicaciones java para acceder a bases de datos relacionales, se trata del API “**JDBC**”
- ▶ La tecnología JDBC dispone de una serie de metodos para operar con la base de datos. Utilizando los mismos, la aplicación dirige todas peticiones hacia un software intermediario, conocido como Driver JDBC.
- ▶ Nos permite independizarnos del tipo de base de datos utilizada para almacenar la información. Con tan solo disponer del driver especifico para el tipo de base de datos con el que se va a trabajar nos basta.

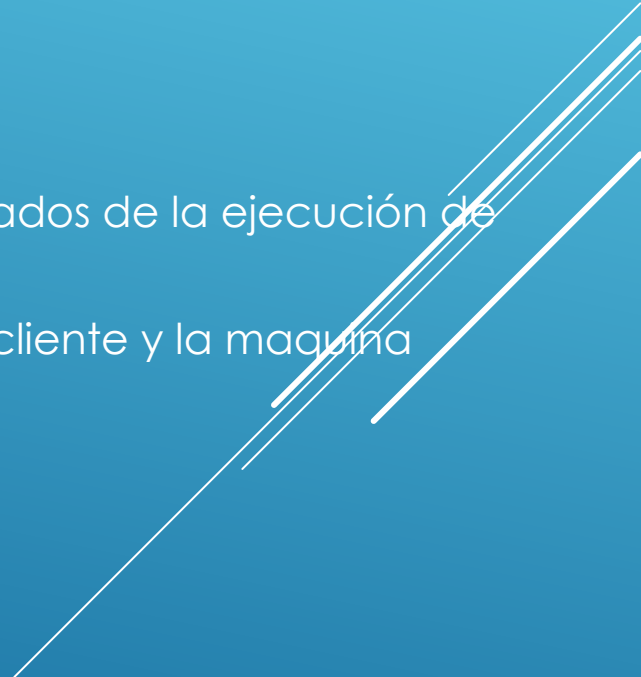
JDBC – JAVA DATABASE CONNECTIVITY

- ▶ Los administradores de bases de datos proveen drivers necesarios para acceder a su base de datos.
 - ▶ Como por ejemplo: Developed by MySQL
- ▶ También existe drivers provistos por terceras partes.
 - ▶ Como por ejemplo: Developed by Community.
- ▶ En un driver JDBC se distinguen dos capas:
 - ▶ Capa de aplicación: la misma interactúa con la aplicación, proporcionan la misma interfaz de aplicación independiente del tipo de base de datos.
 - ▶ Capa de base de datos: la misma interactúa con la base de datos, por lo que es específica para cada tipo de base de datos.

JDBC – JAVA DATABASE CONNECTIVITY

- ▶ Modelos de Aplicaciones:

- ▶ Modelos de dos capas (two-tier):

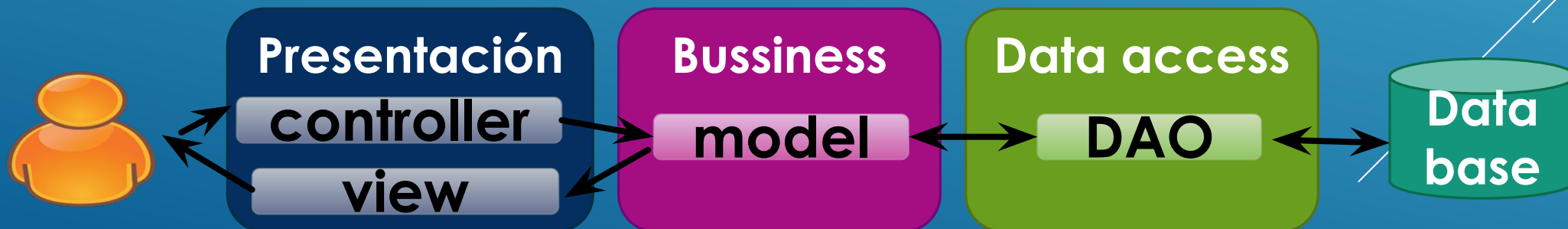
- ▶ La aplicación Java se comunica directamente con la base de datos.
 - ▶ Las sentencias SQL del usuario son enviadas a la base de datos, y los resultados de la ejecución de estas sentencias son devueltas al usuario.
 - ▶ Esta es una arquitectura cliente/servidor, con la máquina del usuario como cliente y la máquina que contiene a la base de datos como servidor.
- 
- Several white lines of varying thickness and length are drawn diagonally across the bottom right corner of the slide, extending from the right edge towards the bottom left.

JDBC – JAVA DATABASE CONNECTIVITY

► Modelos de Aplicaciones:

► Modelos de tres capas (three-tier):

- Los comandos SQL son enviados a una capa intermedia de servicios, la cual envía las sentencias a la base de datos.
- La base de datos procesa las sentencias y devuelve los resultados a la capa intermedia que se los enviara al usuario.
- Ventajas:
 - El usuario puede utilizar una API de mas alto nivel, y por lo tanto mas sencilla de manejar.
 - Sera traducida por la capa intermedia a las llamadas apropiadas, en este caso utilizando el API JDBC.

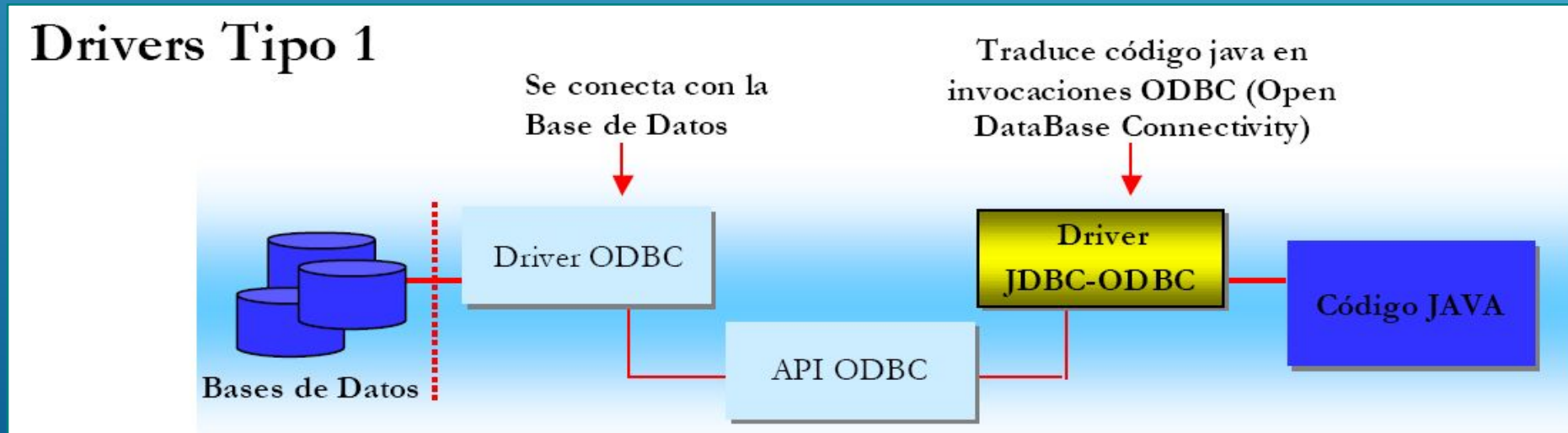


JDBC – JAVA DATABASE CONNECTIVITY

► Tipos de driver JDBC:

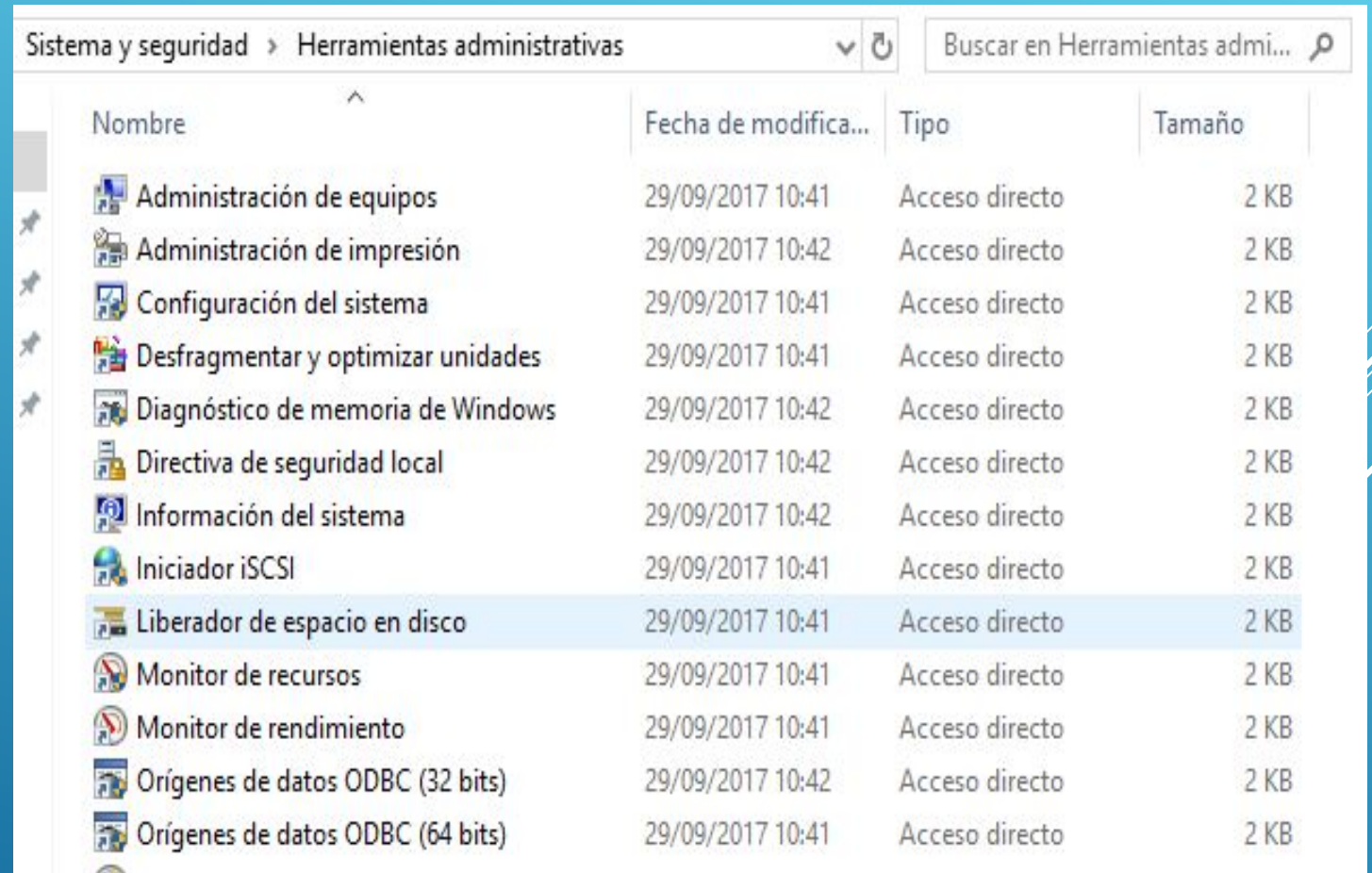
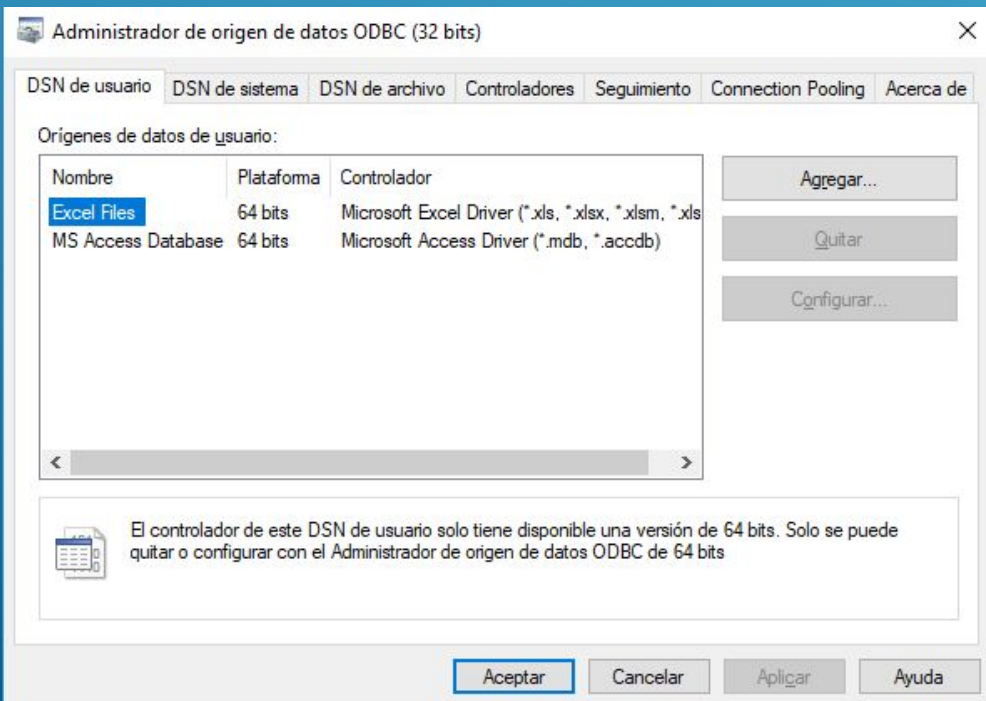
► Tipo 1: Driver puente JDBC-ODBC:

- Fueron los primeros tipos de driver. Se pueden utilizar para varios tipos de base de datos.
- Se puede utilizar cualquier driver ODBC existente.
- La principal desventaja es que hay que configurar un DSN (Data Source Name).
- La configuracion del DSN se debe de hacer desde cada maquina cliente.



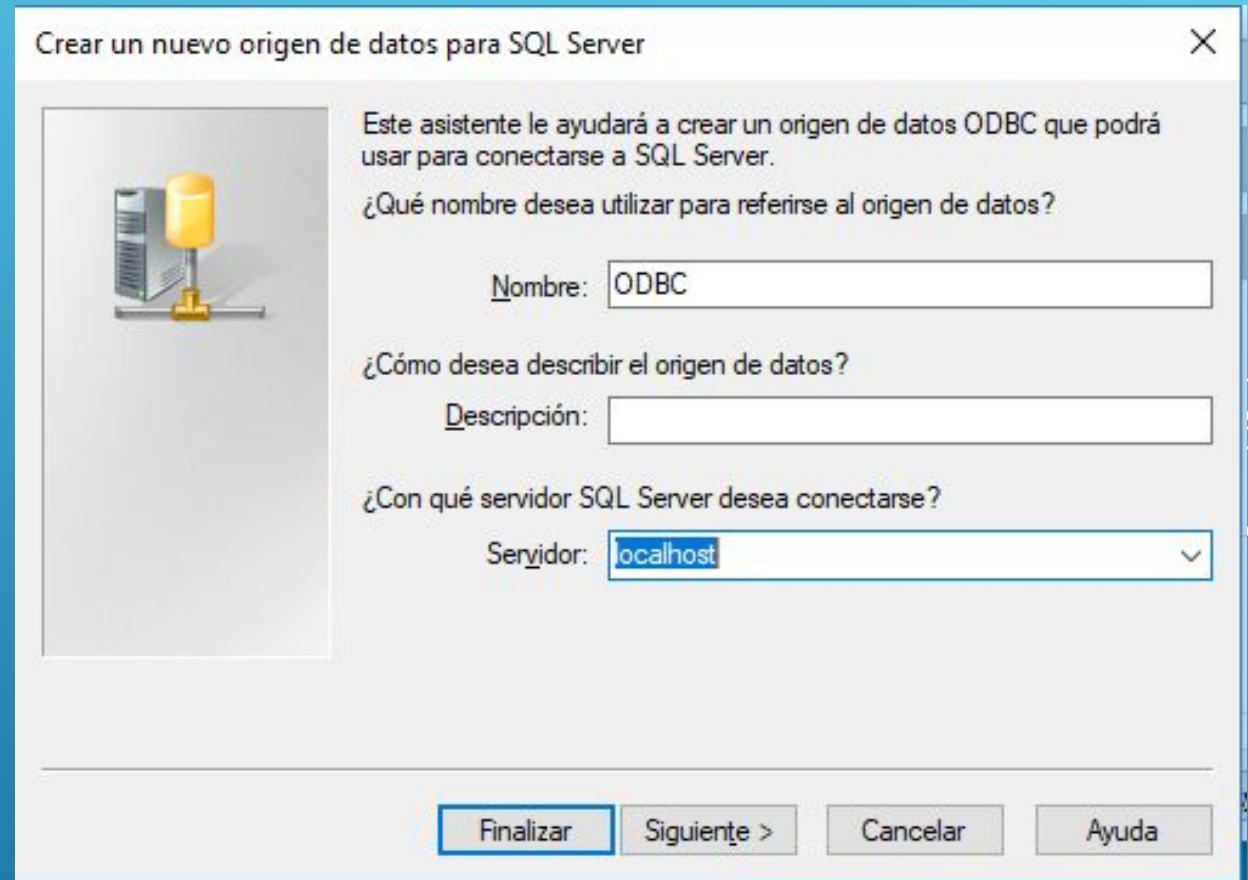
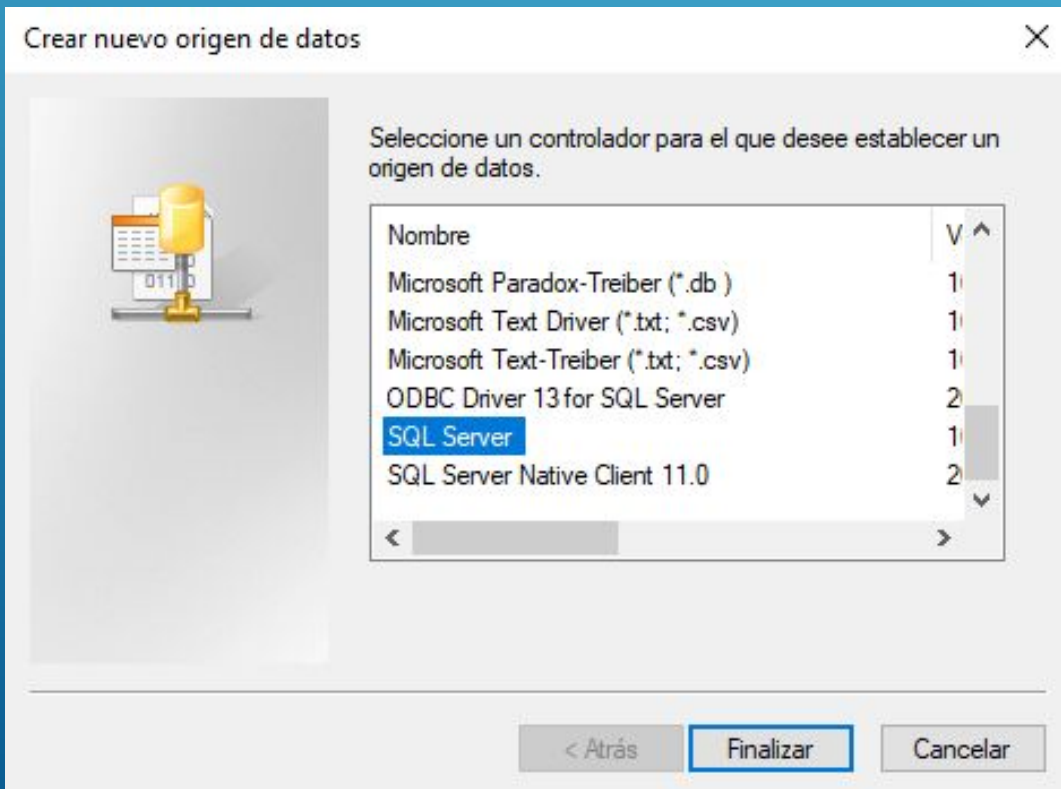
JDBC – JAVA DATABASE CONNECTIVITY

- ▶ Tipos de driver JDBC - ODBC:
 - ▶ 1 - Herramientas Administrativas
 - ▶ 2 – Origen de datos ODBC



JDBC – JAVA DATABASE CONNECTIVITY

- Tipos de driver JDBC - ODBC:
 - 3 – Seleccionar SGBD
 - 4 – Crear ODBC



JDBC – JAVA DATABASE CONNECTIVITY

- Tipos de driver JDBC - ODBC:
 - 5 – Seleccionar inicio de sesión
 - 6 – Seleccionar base de datos

Crear un nuevo origen de datos para SQL Server



☒ Establecer la siguiente base de datos como predeterminada:
master

☐ Adjuntar nombre del archivo de la base de datos:

☒ Usar identificadores entrecomillados ANSI.

☒ Usar nulos, rellenos y advertencias ANSI.

☐ Usar el servidor SQL Server de conmutación por error si el servidor SQL Server principal no se encuentra disponible.

< Atrás **Siguiente >** Cancelar Ayuda

Crear un nuevo origen de datos para SQL Server



¿Cómo desea que SQL Server compruebe la autenticidad del id. de inicio de sesión?

☐ Con la autenticación de Windows NT, mediante el id. de inicio de sesión de red.

☒ Con la autenticación de SQL Server, mediante un id. de inicio de sesión y una contraseña escritos por el usuario.

Para cambiar la biblioteca de red usada para comunicarse con SQL Server, haga clic en Configuración del cliente.

[Configuración del cliente...](#)

☒ Conectar con SQL Server para obtener la configuración predeterminada de las opciones de configuración adicionales.

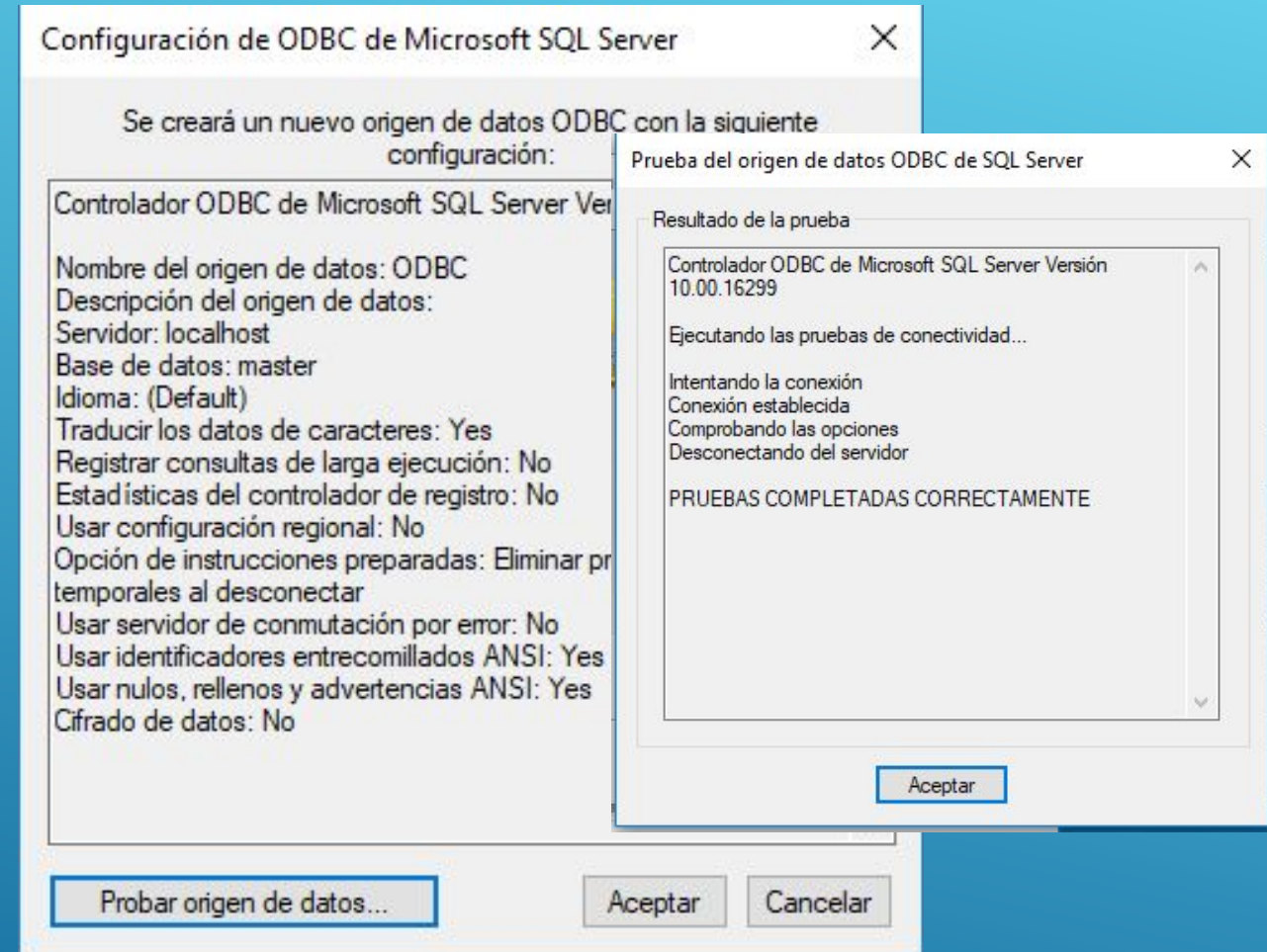
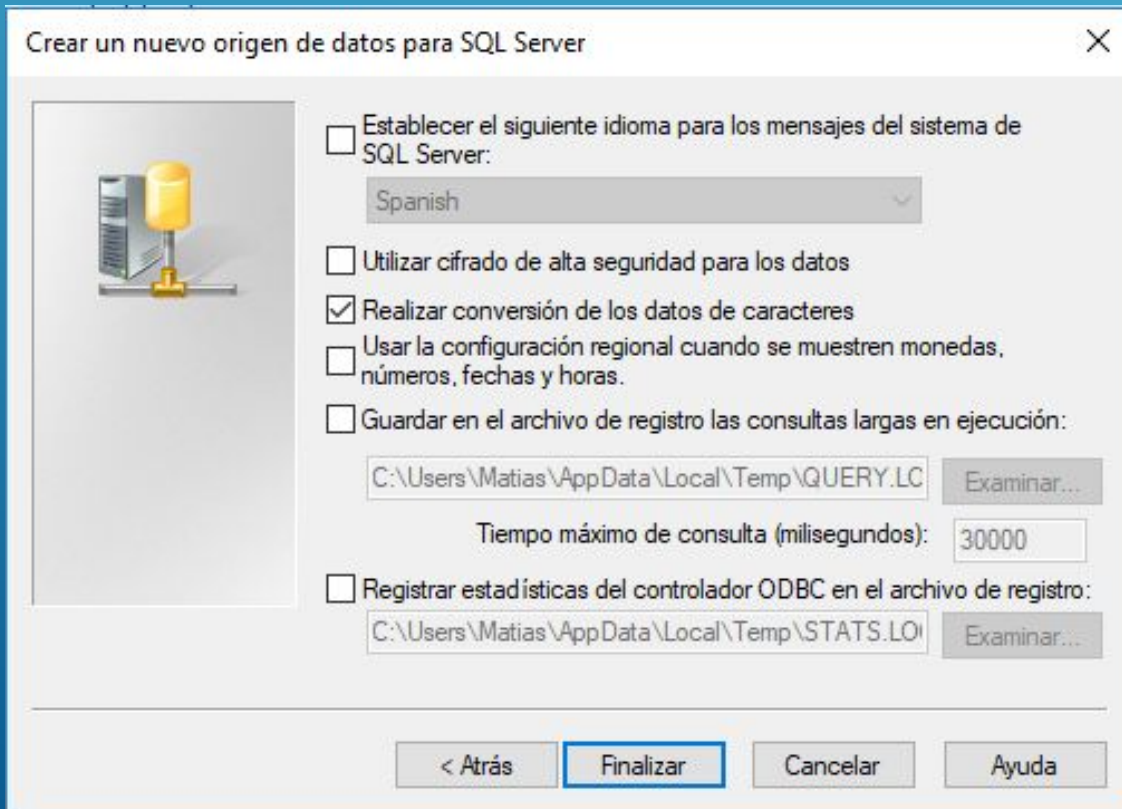
Id. de inicio de sesión:

Contraseña:

< Atrás **Siguiente >** Cancelar Ayuda

JDBC – JAVA DATABASE CONNECTIVITY

- Tipos de driver JDBC - ODBC:
 - 7 – Finalizar
 - 8 – Probar conexión y Aceptar



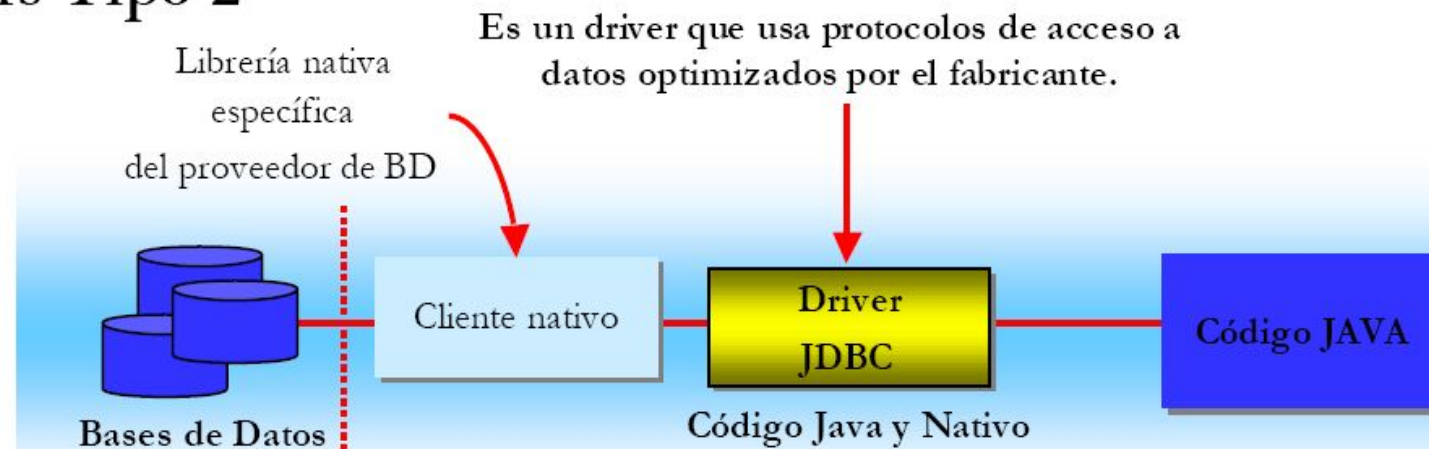
JDBC – JAVA DATABASE CONNECTIVITY

► Tipos de driver JDBC:

► Tipo 2: Driver nativo:

- Convierte las llamadas JDBC en llamadas al API nativo del gestor de base de datos.
- No es un driver 100% Java.
- La principal desventaja es que hay instalar el API nativo de la base de datos en el equipo donde reside la aplicación Java.

Drivers Tipo 2



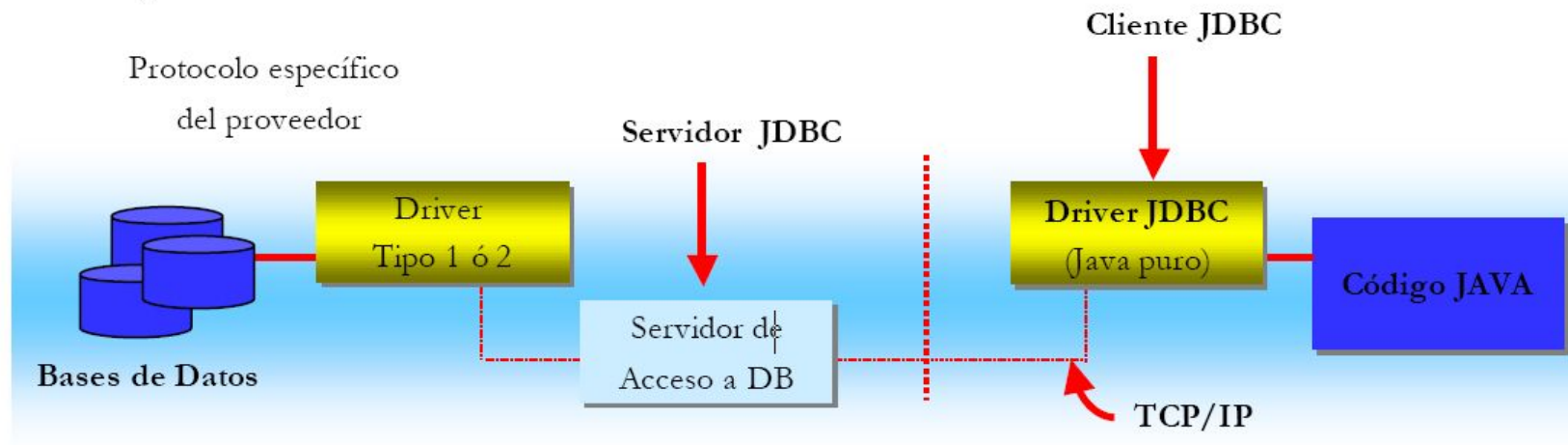
JDBC – JAVA DATABASE CONNECTIVITY

► Tipos de driver JDBC:

► Tipo 3: Driver intermedio:

- Convierte las llamadas JDBC en protocolo específico del middleware, a su vez, éste mismo transforma las llamadas en el API nativo de la base de datos.
- Requiere un middleware server junto con el motor de BD.
- No requiere ningún tipo de instalación en el equipo donde se ejecuta la aplicación.

Drivers Tipo 3



JDBC – JAVA DATABASE CONNECTIVITY

► Tipos de driver JDBC:

► Tipo 4: Driver puro java:

- Convierte las llamadas JDBC en protocolo de red utilizado directamente por el servidor de BD, permitiendo llamadas directas desde la maquina cliente.
- No requiere ningún tipo de configuración especial en la maquina cliente.
- Son altamente portables ya que compilados pueden usarse en cualquier S.O.

Drivers Tipo 4



JDBC – JAVA DATABASE CONNECTIVITY

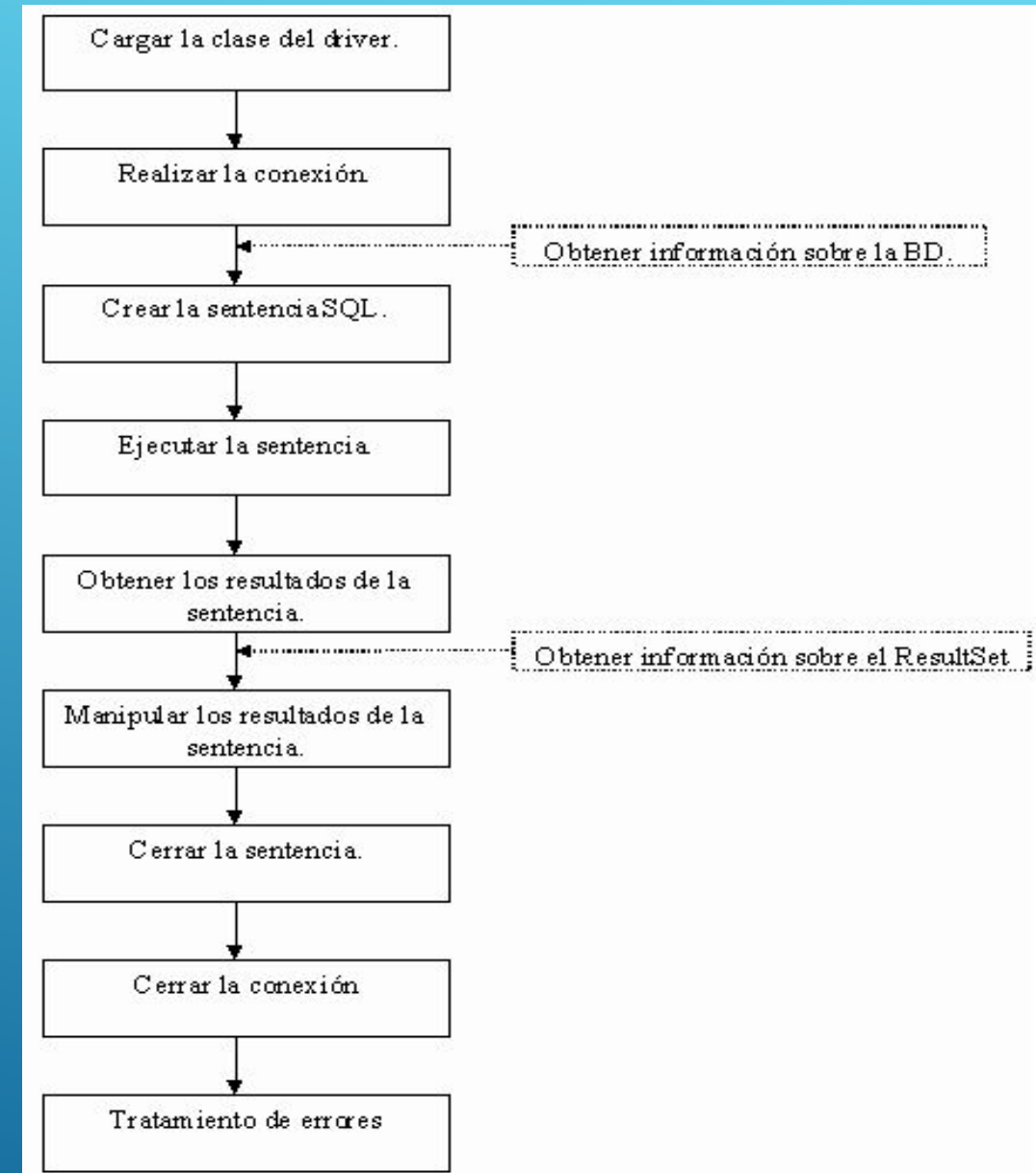
► Tipos de driver JDBC:

- Si se necesita acceder a una base de datos del tipo: Oracle, Sybase, o IBM, el tipo de driver preferible es del **tipo 4**.
- Si la aplicación Java desarrollada accede a varios tipos de base de datos al mismo tiempo, es preferible del **tipo 3**.
- El **tipo 2** se utiliza cuando el tipo 3 o 4 no están disponibles para la base de datos.
- El **tipo 1** se utiliza en general para ambientes de desarrollo o testing.

JDBC – JAVA DATABASE CONNECTIVITY

Ciclo de vida JDBC.

- 1 – Cargar la clase del Driver. (Depende del SGBD)
- 2 – Realizar la conexión. (Mediante la API JDBC)
- 3 – Crear la sentencias SQL (DML)
- 4 – Ejecutar Sentencias. (La API JDBC nos permite tres manera distintas)
- 5 – Obtener los resultados de la sentencia.
- 6 – Manipular los resultados de la sentencia.
- 7 – Cerrar la sentencia
- 8 – Cerrar la conexión.
- 9 – Tratamiento de errores.



JDBC – JAVA DATABASE CONNECTIVITY

▶ API JDBC - Conexión:

- ▶ Para realizar cualquier operación es necesario realizar dos operaciones:
 - ▶ Carga del driver.
 - ▶ Creación de la conexión.

▶ Carga del driver:

- ▶ Se realiza mediante el método estático **forName(String)** de la clase **java.lang.Class**.
- ▶ La clase será cargada en tiempo de ejecución.
- ▶ El método ejecutado inicializa la clase.
- ▶ El método devuelve un objeto asociado a la clase.

JDBC – JAVA DATABASE CONNECTIVITY

▶ API JDBC - Conexión:

▶ Creación de la conexión:

- ▶ La conexión se lleva a cabo con el método **getConnection()** de la clase **DriverManager**.
- ▶ Una sola aplicación puede tener una o mas conexiones con una misma BD, o varias a diferentes BD.
- ▶ El método de conexión devuelve un objeto que implementa la interfaz **Connection**, la cual proporciona varios métodos para manejar la conexión.
- ▶ Este método esta sobrecargado por lo que se puede indicar:
 - ▶ Solamente la URL (Formato: jdbc:subprotocolo:subname)
 - ▶ La URL con el identificador de usuario y contraseña
 - ▶ Ejemplos:
 - **Connection con = DriverManager.getConnection(url);**
 - **Connection con = DriverManager.getConnection(url, usuario, password);**

JDBC – JAVA DATABASE CONNECTIVITY

▶ API JDBC - Conexión:

▶ Creación de la conexión:

▶ Armado de la URL:

- ✓ Varía según el tipo de base de datos
 - ✓ Protocolo de comunicación mas ":" □ "jdbc"
 - ✓ Subprotocolo de comunicación mas ":" □ "sqlserver"
 - ✓ Host de la base de datos (IP) mas ":" □ "//localhost"
 - ✓ Puerto de escucha de la base de datos mas ";" □ "1433"
 - ✓ Nombre de la base de datos □ "databaseName=pubs"
 - ✓ URL completa □ "**jdbc:sqlserver://XXX.XXX.XXX:YYYY;databaseName=pubs**"
- Al finalizar el uso de la base de datos se debe de utilizar el método "**close()**" para liberar los recursos consumidos anteriormente.

JDBC – JAVA DATABASE CONNECTIVITY

► API JDBC – Conexión SQL Server:

```
public static void main(String[] args) {  
    Connection con = null;  
    try {  
        Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver");  
    } catch (ClassNotFoundException e) {  
        e.printStackTrace();  
    }  
  
    try {  
        con = DriverManager.getConnection("jdbc:sqlserver://127.0.0.1:1433;databaseName=ProgAvanzada", "sa", "123456");  
        if(!con.isClosed()) {  
            System.out.println("Conexion establecida.");  
            con.close();  
            System.out.println("Conexion cerrada");  
        }  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

JDBC – JAVA DATABASE CONNECTIVITY

► API JDBC – Validar conexión :

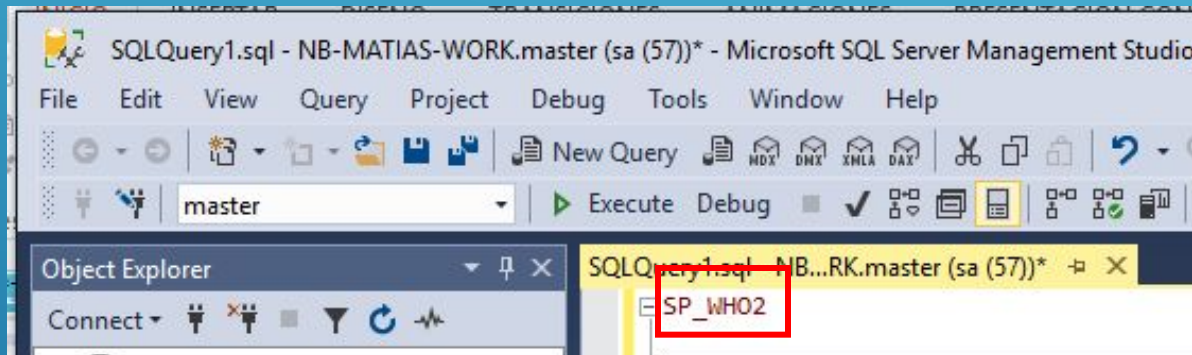
1 - Se agrega un sleep para dejar el hilo de ejecución con la conexión activa el motor de BD

```
try {
    con = DriverManager.getConnection("jdbc:sqlserver://127.0.0.1:1433;databaseName=ProgAvanzada",
    try {
        Thread.sleep(8000);
    } catch (InterruptedException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
    if(!con.isClosed()) {
        System.out.println("Conexion establecida.");
        con.close();
        System.out.println("Conexion cerrada");
    }
} catch (SQLException e) {
    e.printStackTrace();
}
```

JDBC – JAVA DATABASE CONNECTIVITY

► API JDBC – Validar conexión :

2 – A través de la sentencia SP_WHO2 podemos validar la conexión creada desde el hilo principal de la JVM.



The screenshot shows the 'Results' tab of the SQL Server Enterprise Manager. The results grid displays the output of the SP_WHO2 stored procedure, which lists active processes. The row for 'ProgAvanzada' (Microsoft SQL Server JDBC Driver) is highlighted with a red box.

	DBName	Command	CPUTime	DiskIO	LastBatch	ProgramName	SP
46	master	AWAITING COMMAND	0	8	06/24 21:33:38	SQLServerCEIP	51
47	ProgAvanzada	AWAITING COMMAND	0	0	06/24 21:37:40	Microsoft SQL Server JDBC Driver	52
48	ReportServer	AWAITING COMMAND	0	0	06/24 21:37:35	Report Server	53
49	ReportServer	AWAITING COMMAND	0	0	06/24 21:37:24	Report Server	55
50	master	AWAITING COMMAND	0	240	06/24 21:31:12	Microsoft SQL Server Manageme...	56
51	master	SELECT INTO	93	76	06/24 21:37:41	Microsoft SQL Server Manageme...	57